SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering ASSESSMENT

TOOL 2 – ASSIGNMENT

Course Code:	CSA0901	**Course Title:**	**Programming in Java**
CO Assessed:	CO2 – Precision (BL3,BL4)	**Assessment:**	ASSESSMENT TOOL 2 - ASSIGNMENT
Weightage:		**Total Marks:**	100
CO1 Statement:	Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications		
Student Name:		**Reg. No.:**	
Section /Batch:	SLOT B	**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 1 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 30 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. String Analysis System

Develop a Java application to analyze a given paragraph by counting vowels, consonants, digits, words, sentences, spaces, and special characters using String methods.

ANSWER – CODE

```
import java.util.*;
public class StringAnalysis {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int v=0,c=0,d=0,w=0,sent=0,sp=0,special=0;
        for(char ch:s.toCharArray()) {
            if("aeiouAEIOU".indexOf(ch)>=0) v++;
            else if(Character.isLetter(ch)) c++;
            else if(Character.isDigit(ch)) d++;
            else if(ch==' ') sp++;
            else if(ch=='.'||ch=='!'||ch=='?') sent++;
            else special++;
        }
        if(!s.trim().isEmpty()) w=s.trim().split("\\s+").length;
        System.out.println("Vowels: "+v);
        System.out.println("Consonants: "+c);
        System.out.println("Digits: "+d);
        System.out.println("Words: "+w);
        System.out.println("Sentences: "+sent);
        System.out.println("Spaces: "+sp);
        System.out.println("Special characters: "+special);
    }
}
```

INPUT

```
Hello World 123! Java.
```

OUTPUT

```
Vowels: 6
Consonants: 8
Digits: 3
Words: 4
Sentences: 1
Spaces: 3
Special characters: 0
```

Q2. Secure Password Validator

Design a Java program that validates passwords based on length, uppercase letters, lowercase letters, digits, and special characters using String handling techniques.

ANSWER – CODE

```
import java.util.*;
public class PasswordValidator {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String p=sc.nextLine();
        boolean u=false,l=false,d=false,sp=false;
        for(char ch:p.toCharArray()) {
            if(Character.isUpperCase(ch)) u=true;
            else if(Character.isLowerCase(ch)) l=true;
            else if(Character.isDigit(ch)) d=true;
            else sp=true;
        }
        if(p.length()>=8 && u && l && d && sp)
            System.out.println("Valid Password");
        else
            System.out.println("Invalid Password");
    }
}
```

INPUT

```
Java@123
```

OUTPUT

```

Valid Password
```

Q3. Text Compression Utility

Develop a Java application to compress a string by replacing consecutive repeated characters with their frequencies.

ANSWER – CODE

```
import java.util.*;
public class TextCompression {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String s=sc.nextLine();
        StringBuilder out=new StringBuilder();
        for(int i=0;i<s.length();) {
            int j=i;
            while(j<s.length() && s.charAt(j)==s.charAt(i)) j++;
            out.append(s.charAt(i)).append(j-i);
            i=j;
        }
        System.out.println("Compressed: "+out);
    }
}
```

INPUT

```
aaabbccccd
```

OUTPUT

```
Compressed: a3b2c4d1
```

Q4. Student Registration System

Develop a Student Registration System using **HashSet** and **HashMap** with Generics to store unique students, assign courses, and display the data using an Iterator.

ANSWER – CODE

```
import java.util.*;
public class StudentRegistration {
    public static void main(String[] args) {
        HashSet<String> students=new HashSet<>();
        HashMap<String,String> course=new HashMap<>();
        students.add("101-Alex");
        students.add("102-Bob");
        students.add("101-Alex");
        course.put("101-Alex","Java");
        course.put("102-Bob","Python");
        Iterator<String> it=students.iterator();
        while(it.hasNext()) {
            String s=it.next();
            System.out.println(s+" -> "+course.get(s));
        }
    }
}
```

OUTPUT

```
101-Alex -> Java
102-Bob -> Python
```

Q5. Library Book Management System

Develop a Library Management System using **ArrayList** and **HashMap** to manage books, issue books, return books, and search by ISBN.

ANSWER – CODE

```
import java.util.*;
public class LibraryManagement {
    public static void main(String[] args) {
        ArrayList<String> books=new ArrayList<>();
        HashMap<String,String> map=new HashMap<>();
        books.add("Java Basics");
        books.add("Python Guide");
        map.put("97801","Java Basics");
        map.put("97802","Python Guide");
        String isbn="97801";
        System.out.println("Book: "+map.get(isbn));
        System.out.println("Issued: "+map.get(isbn));
        map.remove(isbn);
        System.out.println("Returned successfully");
    }
}
```

OUTPUT

```
Book: Java Basics
Issued: Java Basics
Returned successfully
```

Q6. Employee Management System

Design an Employee Management System using **ArrayList** to store employee details and use an **Iterator** to display employees with salaries greater than a specified amount.

ANSWER – CODE

```
import java.util.*;
public class EmployeeManagement {
    public static void main(String[] args) {
        ArrayList<String> emp=new ArrayList<>();
        emp.add("101 Alice 45000");
        emp.add("102 Bob 60000");
        emp.add("103 Charlie 75000");
        Scanner sc=new Scanner(System.in);
        double limit=sc.nextDouble();
        Iterator<String> it=emp.iterator();
        while(it.hasNext()) {
            String[] a=it.next().split(" ");
            if(Double.parseDouble(a[2])>limit)
                System.out.println(a[0]+" "+a[1]+" "+a[2]);
        }
    }
}
```

INPUT

```
50000
```

OUTPUT

```
102 Bob 60000
103 Charlie 75000
```

Q7. Online Shopping Cart

Develop an Online Shopping Cart using **ArrayList** to add, update, remove, search, and display products. Calculate the total bill.

Develop an Online Shopping Cart using ArrayList to add, update, remove, search, and display products. Calculate the total bill.

ANSWER – CODE

```
import java.util.*;
public class ShoppingCart {
    public static void main(String[] args) {
        ArrayList<String> products=new ArrayList<>();
        HashMap<String,Double> price=new HashMap<>();
        products.add("Pen");
        products.add("Book");
        products.add("Bag");
        price.put("Pen",10.0);
        price.put("Book",50.0);
        price.put("Bag",500.0);
        products.remove("Pen");
        price.put("Book",60.0);
        double total=0;
        for(String p:products) total+=price.get(p);
        System.out.println("Products: "+products);
        System.out.println("Total Bill: Rs."+total);
    }
}
```

OUTPUT

```
Products: [Book, Bag]
Total Bill: Rs.560.0
```

Q8. Hospital Patient Management System

Develop a Hospital Patient Management System using **HashMap** to map patient IDs with patient details and use an **Iterator** to display all records.

ANSWER – CODE

```
import java.util.*;
public class HospitalPatient {
    public static void main(String[] args) {
        HashMap<Integer,String> patients=new HashMap<>();
        patients.put(101,"Arun - Fever");
        patients.put(102,"Bala - Cold");
        patients.put(103,"Charan - Fracture");
        Iterator<Map.Entry<Integer,String>> it=patients.entrySet().iterator();
        while(it.hasNext()) {
            Map.Entry<Integer,String> e=it.next();
            System.out.println(e.getKey()+" -> "+e.getValue());
        }
    }
}
```

OUTPUT

```
101 -> Arun - Fever
102 -> Bala - Cold
103 -> Charan - Fracture
```

Q9. Banking Account Management System

Develop a Banking System using **HashMap** to maintain account details. Implement deposit, withdrawal, balance inquiry, and account search operations.

ANSWER – CODE

```
import java.util.*;
public class BankingSystem {
    public static void main(String[] args) {
        HashMap<Integer,Double> acc=new HashMap<>();
        acc.put(1001,5000.0);
        acc.put(1002,3000.0);
        int id=1001;
        double deposit=1000, withdrawal=500;
        acc.put(id,acc.get(id)+deposit);
        acc.put(id,acc.get(id)-withdrawal);
        System.out.println("Balance: Rs."+acc.get(id));
        System.out.println("Account found: "+acc.containsKey(id));
    }
}
```

OUTPUT

```
Balance: Rs.5500.0
Account found: true
```

Q10. Course Enrollment System

Design a Course Enrollment System using **HashMap** and **ArrayList** to maintain courses and enrolled students.

ANSWER – CODE

```
import java.util.*;
public class CourseEnrollment {
    public static void main(String[] args) {
        HashMap<String,ArrayList<String>> courses=new HashMap<>();
        courses.put("Java",new ArrayList<>());
        courses.put("Python",new ArrayList<>());
        courses.get("Java").add("Arun");
        courses.get("Java").add("Bala");
        courses.get("Python").add("Charan");
        for(String c:courses.keySet())
            System.out.println(c+" -> "+courses.get(c));
    }
}
```

OUTPUT

```
Java -> [Arun, Bala]
Python -> [Charan]
```

Q11. Phone Directory Application

Develop a Phone Directory using **HashMap** to store contact names and phone numbers. Implement add, update, delete, and search operations.

ANSWER – CODE

```
import java.util.*;
public class PhoneDirectory {
    public static void main(String[] args) {
        HashMap<String,String> phone=new HashMap<>();
        phone.put("Arun","9876543210");
        phone.put("Bala","9123456780");
        phone.put("Arun","9000000000");
        phone.remove("Bala");
        String name="Arun";
        System.out.println(name+" -> "+phone.get(name));
    }
}
```

OUTPUT

```
Arun -> 9000000000
```

Q12. Vehicle Registration System

Develop a Vehicle Registration System using **HashSet** to maintain unique vehicle registration numbers and display them using an Iterator.

ANSWER – CODE

```
import java.util.*;
public class VehicleRegistration {
    public static void main(String[] args) {
        HashSet<String> vehicles=new HashSet<>();
        vehicles.add("TN01AB1234");
        vehicles.add("TN02CD5678");
        vehicles.add("TN01AB1234");
        Iterator<String> it=vehicles.iterator();
        while(it.hasNext()) System.out.println(it.next());
    }
}
```

OUTPUT

```
TN01AB1234
TN02CD5678
```

Q13. Hotel Reservation System

Develop a Hotel Reservation System using **HashMap** to map room numbers with guest details and implement room booking and cancellation.

ANSWER – CODE

```
import java.util.*;
public class HotelReservation {
    public static void main(String[] args) {
        HashMap<Integer,String> rooms=new HashMap<>();
        rooms.put(101,"Arun");
        rooms.put(102,"Bala");
        int room=101;
        System.out.println("Room "+room+" booked by "+rooms.get(room));
        rooms.remove(room);
        System.out.println("Room "+room+" cancelled");
    }
}
```

OUTPUT

```
Room 101 booked by Arun
Room 101 cancelled
```

Q14. Online Quiz Management System

Develop a Quiz Management System using **HashMap** to store questions and answers and display them using an Iterator.

ANSWER – CODE

```
import java.util.*;
public class OnlineQuiz {
    public static void main(String[] args) {
        HashMap<String,String> q=new HashMap<>();
        q.put("Capital of India?","Delhi");
        q.put("2 + 2 = ?","4");
        Iterator<Map.Entry<String,String>> it=q.entrySet().iterator();
        while(it.hasNext()) {
            Map.Entry<String,String> e=it.next();
            System.out.println("Q: "+e.getKey());
            System.out.println("A: "+e.getValue());
        }
    }
}
```

OUTPUT

```
Q: Capital of India?
A: Delhi
Q: 2 + 2 = ?
A: 4
```

Q15. Student Marks Management System

Develop a Student Marks Management System using **HashMap** to store student details and calculate grades based on marks.

ANSWER – CODE

```
import java.util.*;
public class StudentMarks {
    public static void main(String[] args) {
        HashMap<String,Integer> marks=new HashMap<>();
        marks.put("Arun",85);
        marks.put("Bala",72);
        marks.put("Charan",55);
        for(String name:marks.keySet()) {
            int m=marks.get(name);
            String grade=m>=90?"A+":m>=80?"A":m>=70?"B":m>=60?"C":"D";
            System.out.println(name+" -> "+m+" -> Grade "+grade);
        }
    }
}
```

OUTPUT

```
Arun -> 85 -> Grade A
Bala -> 72 -> Grade B
Charan -> 55 -> Grade D
```

Q16. Attendance Management System

Develop a Student Attendance Management System using **HashMap** to maintain attendance records and calculate attendance percentages.
calculate attendance percentages.

ANSWER – CODE

```
import java.util.*;
public class AttendanceManagement {
    public static void main(String[] args) {
        HashMap<String,int[]> data=new HashMap<>();
        data.put("Arun",new int[]{18,20});
        data.put("Bala",new int[]{15,20});
        for(String name:data.keySet()) {
            int[] a=data.get(name);
            double p=a[0]*100.0/a[1];
            System.out.printf("%s -> %.2f%%%n",name,p);
        }
    }
}
```

OUTPUT

```
Arun -> 90.00%
Bala -> 75.00%
```

Q17. Inventory Management System

Develop an Inventory Management System using **HashMap** to store product information and update stock quantities after sales.

ANSWER – CODE

```
import java.util.*;
public class InventoryManagement {
    public static void main(String[] args) {
        HashMap<String,Integer> stock=new HashMap<>();
        stock.put("Pen",100);
        stock.put("Book",50);
        stock.put("Bag",20);
        String product="Book";
        int sold=5;
        stock.put(product,stock.get(product)-sold);
        for(String p:stock.keySet())
            System.out.println(p+" -> "+stock.get(p));
    }
}
```

OUTPUT

```
Pen -> 100
Book -> 45
Bag -> 20
```

Q18. Payroll Management System

Develop a Payroll Management System using **ArrayList** to store employee records and calculate net salary using Iterators.

NSWER – CODE

```
import java.util.*;
public class Payroll {
    public static void main(String[] args) {
        ArrayList<double[]> emp=new ArrayList<>();
        emp.add(new double[]{101,30000,2000,1000});
        emp.add(new double[]{102,40000,3000,1500});
        Iterator<double[]> it=emp.iterator();
        while(it.hasNext()) {
            double[] e=it.next();
            double net=e[1]+e[2]-e[3];
            System.out.println("Employee "+(int)e[0]+" Net Salary: Rs."+net);
        }
    }
}
```

OUTPUT

```
Employee 101 Net Salary: Rs.31000.0
Employee 102 Net Salary: Rs.41500.0
```

Q19. Generic Student Repository

Design a Generic Repository using **Generics** to store and retrieve student, faculty, and course objects.

ANSWER – CODE

```
import java.util.*;
class Repository<T> {
    private ArrayList<T> list=new ArrayList<>();
    void add(T obj){ list.add(obj); }
    void display(){ for(T x:list) System.out.println(x); }
}
public class GenericRepository {
    public static void main(String[] args) {
        Repository<String> students=new Repository<>();
        Repository<String> faculty=new Repository<>();
        Repository<String> courses=new Repository<>();
        students.add("Student: Arun");
        faculty.add("Faculty: Kumar");
        courses.add("Course: Java");
        students.display();
        faculty.display();
        courses.display();
    }
}
```

OUTPUT

```
Student: Arun
Faculty: Kumar
Course: Java
```

Q20. Generic Calculator

Develop a Generic Java program that performs addition, subtraction, multiplication, and comparison for different numeric data types.

ANSWER – CODE

```
import java.util.*;
class Calculator<T extends Number> {
    double add(T a,T b){ return a.doubleValue()+b.doubleValue(); }
    double sub(T a,T b){ return a.doubleValue()-b.doubleValue(); }
    double mul(T a,T b){ return a.doubleValue()*b.doubleValue(); }
    int compare(T a,T b){ return Double.compare(a.doubleValue(),b.doubleValue()); }
}
public class GenericCalculator {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        double a=sc.nextDouble(), b=sc.nextDouble();
        Calculator<Double> c=new Calculator<>();
        System.out.println("Addition: "+c.add(a,b));
        System.out.println("Subtraction: "+c.sub(a,b));
        System.out.println("Multiplication: "+c.mul(a,b));
```

```
        System.out.println("Comparison: "+(c.compare(a,b)>0?"First is greater":
                c.compare(a,b)<0?"Second is greater":"Both are equal"));
    }
}
```

INPUT

```
10 5
```

OUTPUT

```
Addition: 15.0
Subtraction: 5.0
Multiplication: 50.0
Comparison: First is greater
```

Q21. Book Catalog System

Develop a Book Catalog System using **TreeSet** to store books in alphabetical order and display them using an Iterator.

Q22. Event Registration System

Develop an Event Registration System using **HashSet** to prevent duplicate participant registrations.

Q23. Hospital Appointment Scheduler

Develop a Hospital Appointment Scheduler using **HashMap** to assign doctors to patients and display appointment details.

Q24. Food Delivery Management System

Develop a Food Delivery Management System using **ArrayList** to manage customer orders and calculate the final bill.

Q25. Movie Ticket Booking System

Develop a Movie Ticket Booking System using **HashMap** to manage seat reservations and prevent duplicate bookings.

Q26. Generic Box Class

Implement a Generic `Box<T>` class to store and retrieve different types of objects.

Q27. Online Examination System

Develop an Online Examination System using **ArrayList**, **HashMap**, and **Iterator** to manage students, questions, and scores.

Q28. College Information Management System

Develop a College Information Management System using Collections to manage students, faculty, departments, and courses.

Q29. E-Commerce Product Catalog

Develop an E-Commerce Product Catalog using **HashMap** to store product IDs and product details. Implement search and update operations.

Q30. Personal Expense Tracker

Develop a Personal Expense Tracker using **ArrayList** to record daily expenses, categorize them, and generate monthly expense reports.

Q31. Digital Library System

Develop a Digital Library System using **HashMap**, **ArrayList**, and **Iterator** to manage books and members.

Q32. Customer Feedback Analyzer

Develop a Java application to analyze customer feedback using String Handling and display the frequency of each word using a **HashMap**.

Q33. Word Frequency Counter

Develop a Java program to count the frequency of every word in a paragraph using **HashMap** and **StringTokenizer**.

Q34. Student Result Analyzer

Develop a Java application to calculate class average, highest mark, lowest mark, and grade distribution using Collections.

Q35. Employee Department Mapping

Develop an Employee Department Mapping System using **HashMap** to associate employees with departments and display records using an Iterator.

Q36. Iterator-Based Inventory System

Develop an Inventory System where products can be safely removed during traversal using an **Iterator**.

Q37. Generic Inventory Manager

Develop a Generic Inventory Manager capable of storing different product types using Generics.

Q38. Online Voting System

Develop an Online Voting System using **HashSet** to prevent duplicate voting and **HashMap** to count votes.

Q39. University Course Management System

Develop a University Course Management System using **ArrayList**, **HashMap**, **Iterator**, and **Generics** to manage students, faculty, and courses.

Q40. Integrated Student Information System

Develop a menu-driven Student Information System integrating **String Handling**, **Collections**, **Iterator**, and **Generics**. Implement CRUD operations and generate summary reports.